

Introduction to Deep Learning

Session 10: CNNs in Practice

May 2026

Magda Gregorová - magda.gregorova@thws.de



Licensed according to CC-BY-SA 4.0

Lecture Overview

1. Transfer Learning
2. Practical CNN Training
3. Object Detection
4. Segmentation
5. Summary

Three Regimes

Regime	When	Data needed	Compute
Train from scratch	Novel architecture, very different domain	Large labelled dataset	Very high
Pretrain + fine-tune	Moderate dataset, research setting	Moderate	Medium
Download & adapt	Most practical tasks	Small-medium	Low

Modern workflow: download pretrained model from HuggingFace / torchvision / timm, adapt to task

Training from scratch — rare, reserved for large organisations with compute and data budget

Why Pretrained Models Work

Key insight: CNN layers learn features of increasing specificity

Early layers — general:

- Edges, colours, orientations
- Useful across almost all visual tasks
- Transfer well to new domains

Later layers — specific:

- Object parts, textures, patterns
- Specific to source task
- Need adaptation for new task

Pretrained on: ImageNet (1.2M images), LAION-5B (5.8B image-text pairs), proprietary datasets

Works because: low-level visual features are universal — edges and textures appear in medical images, satellite imagery, microscopy

Feature Extraction vs Fine-tuning

Feature extraction:

- Freeze all pretrained layers
 - Replace and train FC head only
- + Fast, few trainable parameters
+ Works with small datasets
+ No risk of catastrophic forgetting
- Limited adaptation to new domain

Fine-tuning:

- Unfreeze some or all layers
 - Train with small learning rate
- + Better performance
+ Adapts to new domain
- Needs more data
- Risk of catastrophic forgetting

Catastrophic forgetting: large learning rate overwrites pretrained weights — use 10×–100× smaller learning rate than original training

When to Use Which

	Similar domain	Different domain
Small dataset	Feature extraction (freeze all, train head)	Feature extraction (try different layer depths)
Large dataset	Fine-tune all layers (small learning rate)	Fine-tune all layers (more epochs, careful tuning)

Rule of thumb:

- Less data → freeze more layers
- More similar to source domain → reuse more layers
- Always start with feature extraction, add fine-tuning if needed

Freezing strategy:

- Start fully frozen — train head only
- Unfreeze progressively from top (later layers first)
- Earlier layers = more general = freeze longer

Learning rates:

- New head: standard learning rate (e.g. $1e-3$)
- Fine-tuned layers: much smaller (e.g. $1e-5$)
- **Discriminative learning rates** — different rates per layer group
- Warmup before fine-tuning improves stability

Other considerations:

- More aggressive data augmentation when dataset is small
- Early stopping essential — fine-tuning overfits quickly
- Monitor validation loss, not training loss

Two main sources of pretrained models:

`torchvision.models:`

- Official PyTorch model zoo
- ResNet, VGG, EfficientNet, ViT
- Pretrained on ImageNet
- Simple API

```
model = torchvision.models  
.resnet50(weights='DEFAULT')
```

`timm (PyTorch Image Models):`

- 700+ pretrained models
- State-of-the-art architectures
- Multiple pretraining datasets
- Community standard

```
import timm  
model = timm.create_model(  
    'resnet50', pretrained=True)
```

HuggingFace: transformers library for vision-language models (CLIP, ViT, DINO)

Adapting Pretrained Model

Replace classifier head:

```
model = torchvision.models.resnet50(weights='DEFAULT')
# freeze all layers
for param in model.parameters():
    param.requires_grad = False
# replace FC head
num_features = model.fc.in_features
model.fc = nn.Linear(num_features, num_classes)
# only head parameters are trainable
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

For fine-tuning: unfreeze layers selectively

```
# unfreeze last ResNet block
for param in model.layer4.parameters():
    param.requires_grad = True
```

Standard training pipeline:

```
from torchvision import transforms

train_transform = transforms.Compose([
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(
        brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225])
])

val_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(...)
])
```

Note: normalisation values are ImageNet statistics — use same values for pretrained models

Image classification: one label per image

Richer tasks:

Object detection

- Locate and classify multiple objects
- Output: bounding boxes + labels

Semantic segmentation

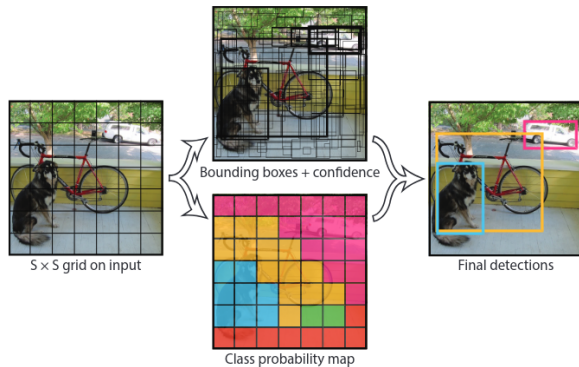
- Classify every pixel
- Output: label map same size as input

Instance segmentation

- Separate instances of same class
- Output: per-instance masks

All built on CNN backbones — pretrained feature extractors + task-specific heads

You Only Look Once — real-time object detection

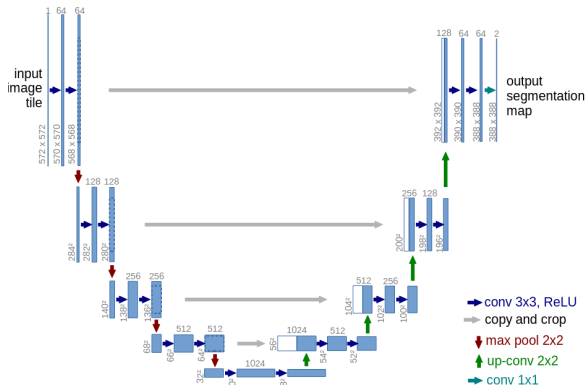


Key idea:

- Divide image into $S \times S$ grid
- Each cell predicts bounding boxes + confidence + class probabilities
- Single forward pass — very fast

Output: $S \times S \times (B \times 5 + C)$ tensor
 B boxes, C classes per cell

Now at v10 — state of the art for real-time detection



Encoder-decoder architecture:

- **Encoder:** conv blocks + pooling, extract features, reduce resolution
- **Decoder:** upsample + conv, recover resolution
- **Skip connections:** concatenate encoder features to decoder — recover spatial detail

Output: pixel-wise label map, same size as input

Originally for medical image segmentation — now widely used

Semantic vs Instance Segmentation

Semantic segmentation:

- Every pixel assigned class label
- All instances of same class merged
- Cannot distinguish individual objects
- U-Net, DeepLab, SegFormer

Instance segmentation:

- Separate mask per object instance
- Distinguishes individual objects
- Combines detection + segmentation
- Mask R-CNN, YOLACT

Panoptic segmentation: unified — semantic for background, instance for objects — modern standard

Transfer learning:

- Download pretrained model
- Feature extraction vs fine-tuning
- Discriminative learning rates
- Freeze progressively

Practical tools:

- `torchvision.models`
- `timm`
- HuggingFace
- ImageNet normalisation

Beyond classification:

- Detection: YOLO, bounding boxes
- Segmentation: U-Net, pixel labels
- Instance: Mask R-CNN
- All use CNN backbones

Next: L11 — Sequence Models & RNNs